A07｜怎样知道这次修改真的修好了？

有返回值不等于结果正确；修好一个案例不等于旧功能没被破坏。**从要求推出预期 → 比较实际值 → 定位最早差异 → 修改 → 重跑原检查。**

测试写什么，工具增加什么？

当前问题	关系或短用法	记到什么程度
预期从哪里来？	九点半应为 9 × 60 + 30 = 570。预期来自换算要求，不是抄程序输出。	**先有判断依据，再让工具比较。**
怎样把要求写成检查？	`assert actual == 570` 检查条件；为假时发出 `AssertionError`，不替程序修正数值。	用熟 assert 实际结果 == 预期结果；比较用 ==，赋值用 =。
pytest 是“激活 assert”吗？	不是。`assert` 属于 Python；pytest 找到测试、调用测试、汇总结果，并增强差异报告。	**断言提出要求，pytest 组织执行。** 定义测试函数不等于已调用。
报告指向 assert，就该改测试吗？	那可能只是发现差异的位置。先核对预期，再沿调用追进业务函数，检查输入与计算。	不为变绿而把正确预期改成错误输出。

这个独立短例把两份工作放在一起展示：

> Python 示例

```
def to_minutes(hours, minutes):
    return hours * 60 + minutes

assert to_minutes(9, 30) == 570
```

函数产生实际值，断言核对它；`570` 来自要求，不是固定语法。若把函数误改为只返回 `hours * 60`，整点 `(9, 0)` 仍能通过，非整点 `(9, 30)` 才暴露遗漏。选择案例是为了让不同错误显现，不是随意多试几个数字。正式项目通常分别保存可复用业务函数与测试。

要实际重复这些检查时，再查运行入口 `python -m pytest test_time_slots.py -q`：让所选 Python 运行 pytest，文件名选目标，`-q` 减少报告细节。它要求当前环境已经安装 pytest、当前位置能找到测试文件；原章提供自己的 `.venv` 入口。命令逐渐用熟，不在值与预期还没分清时先背环境配方。

回看：「九点算对了，九点半就一定算对了吗？」「检查多起来，谁负责一个个调用？」「报告指向 assert，是测试文件写错了吗？」。

测试、业务校验和排错，各管哪一段?

当前问题	关系或短用法	记到什么程度
能依赖 assert 拦截正式业务坏输入吗?	不应依赖它：优化运行可能移除断言。业务规则用普通 `if`；约定拒绝时，可用 `raise ValueError(...)` 发出异常。	**测试检查要求，业务代码执行规则。** `raise` 发出异常，`except` 处理异常。
本来就该报错，怎样测试通过?	`with pytest.raises(ValueError):` 检查块内调用是否发生预期异常；没发生或发生不匹配异常，都不符合预期。	认得用途；完整写法查「不合法的时间本来就该失败，怎样测试?」。
打印对了，返回值为什么不对?	`print` 交给输出通道；`return` 交给调用者。只打印、没返回业务值时，调用结果可能是 `None`。	屏幕看见结果与调用者收到结果分别看。
怎样选案例，而不穷举全部输入?	选择能暴露不同遗漏的情况：整点、非整点、实际范围边界和约定拒绝路线。修复后也重跑原来正确的案例。	不背案例数字、不凑数量；通过只支持实际检查过的情况。
仍找不到原因，看什么?	最小失败调用、临时值输出或断点帮助定位；回溯是异常传出的调用路径，不是全部执行历史。日志留下以后可查的线索。	已定位就修复复查；需要暂停看值时查「值太多时，怎样暂停下来逐步看?」。日志不记录秘密。

重点不是报告格式，而是**要求是否正确、实际值在哪里偏离、原检查是否真的重跑过**。这也解释了为什么只复制一份通过的测试，不足以说明已经理解业务规则。